# Natural Language Processing | NLP : CS60075

## ASSIGNMENT - 3: Implementing BERT from Scratch for Fake News Classification

**Name : Tuhin Mondal** **Roll No: 22CS10087**

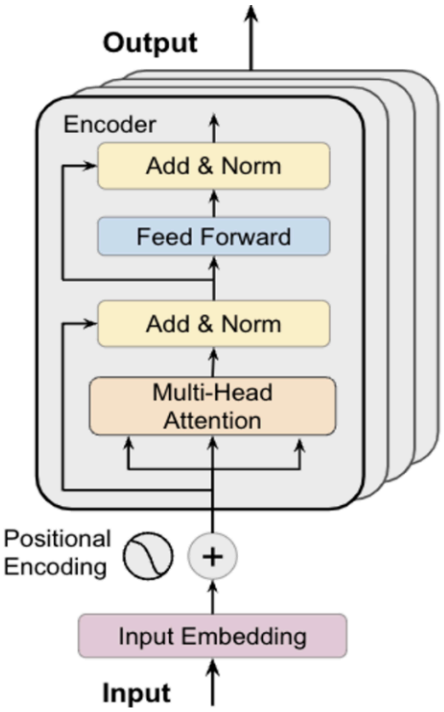
## 1) Overview

This report summarizes the model architecture, hyperparameters, dataset preparation, and final performance of the **BERT-based text classification model** used for the Fakeddit fake/real post detection task. The core objective was twofold: first, to design a BERT model from scratch, and second, to fine-tune a powerful Transformer-based model (BERT) on this enriched data to achieve high classification performance.

## 2) Task - 1: Implement BERT from Scratch

### 2.1) Model Architecture:

The solution uses a custom-built, simplified BERT-like model (**MyBERTModel**) based on PyTorch, leveraging the Transformer Encoder structure to capture contextual relationships within the text. The model is a stacked encoder with an embedding layer, followed by two identical transformer layers, and a final pooling + classification head.




**Key Parameter Shapes (Weight Matrices)**

The shapes of the primary trainable weight matrices reflect the hidden dimensions (H=768) and the intermediate feed-forward dimension (I = 4 x H = 3072):

**Token Embeddings Matrix:** (30522 x 768)
**Query, Key, Value Matrices (in MHA):** (768 x 768) each
**Feed-Forward Layer 1 Weight:** (3072 x 768)
**Feed-Forward Layer 2 Weight:** (768 x 3072)
**Classifier Linear Layer Weight:** (2 x 768)

### 2.2) Key Hyperparameters (Fixed Model Configuration)

The following core dimensions define the capacity and complexity of the model, inherited from the underlying bert-base-uncased design philosophy:

| Parameter | Symbol | Value | Description |
|---|---|---|---|
| **Vocabulary Size** | V | 30,522 | The total no of unique tokens the model can recognize. |
| **Hidden Size** | H | 768 | The dimensionality of the embedding vector space d |
| **Number of Layers** | L | 2 | The number of stacked Encoder blocks used. |
| **Attention Head Counts** | A | 12 | The no of parallel attention heads in the Attention layer. |
| **Intermediate Size** | I | 4 x 768 = 3072 | Size of the hidden layer in Feed-Forward Network (FFN). |
| **Max Sequence Length** | S | 512 | The maximum number of tokens the model can process. |
| **Number of Classes** | C | 2 | The number of output classes: 0 (Real) and 1 (Fake). |

### 2.3) Total Trainable Parameters Calculation

Based on the dimensions defined above, the total number of trainable parameters for the custom BERT architecture L = 2 layers is calculated as follows, accounting for weights and biases:

| Component | Calculation Formula (Weights + Biases) | Parameters (Approx.) |
|---|---|---|
| **Embeddings** | (V+S+T) x H + 4 x H | 23,843,184 |
| **Single Encoder Layer** | 4 x (H x H + H) + 2 x (H x I + I) + 4 x H | 7,091,328 |
| **Total Encoder Layers (**L = 2**)** | 2 x ({Single Layer Parameters}) | 14,182,656 |
| **Pooler Layer** | H x H + H | 590,592 |
| **Classifier Head** | H x C + C | 1,538 |
| **TOTAL** | Sum of all components | **38,617,970** |

### 2.4) Intermediate Data Shapes (Embeddings)

The model begins by converting input token IDs into contextualized embeddings by summing three distinct embedding types: Token, Positional, and Token Type. The combined embedding undergoes Layer Normalization and Dropout before entering the encoder stack. Here **B = Batch Size, S = Max Seq Length**

| Component | Input Shape | Output Shape | Description |
|---|---|---|---|
| **Token Embeddings** | (B, S) | (B, S, 768) | Maps token IDs to vectors. |
| **Positional Embeddings** | (1, S) | (1, S, 768) | Encodes the absolute position of tokens. |
| **Combined Input Embedding** | - | (B, S, 768) | The input to the first Encoder layer. |
| **Output of Multi-Head Attention** | (B, S, 768) | (B, S, 768) | Outputs the attention head |
| **Output of Feed-Forward Layer** | (B, S, 768) | (B, S, 768) | Output after passing through FFN |
| **Output of Transformer Encoder** | (B, S, 768) | (B, S, 768) | The output of one full layer (MHSA + FFN) |
| **Output Probabilities** | - | (B, 2) | Final probability over 2 classes (Real/Fake). |

## 3) Task 2: Pre-Processing and Feature Preparation

### 3.1) Dataset Assembly and Cleaning

**The raw data was collected from three TSV files and concatenated: all_train.tsv, all_validate.tsv, and all_test_public.tsv. Additionally, the large all_comments.tsv file was loaded separately to construct the conversational context feature.**

```
[+] Loading /kaggle/input/fakeddit-nlp-assignment/all_train.tsv

 [+] Required col found: id
 [+] Required col found: clean_title
 [+] Required col found: 2_way_label
 [+] Loaded 878218 rows.


[+] Loading /kaggle/input/fakeddit-nlp-assignment/all_validate.tsv

 [+] Required col found: id
 [+] Required col found: clean_title
 [+] Required col found: 2_way_label
 [+] Loaded 92444 rows.
```

```
[+] Loading /kaggle/input/fakeddit-nlp-assignment/all_test_public.tsv

 [+] Required col found: id
 [+] Required col found: clean_title
 [+] Required col found: 2_way_label
 [+] Loaded 92444 rows.

[+] df1 shape: (878218, 3)
[+] df2 shape: (92444, 3)
[+] df3 shape: (92444, 3)

[+] Combined shape (before cleaning): (1063106, 3)
[+] NaN counts: 91300

[+] Dropped 91300 rows containing NaNs
[+] Combined shape (after cleaning): (971806, 3)
```

**Initial Combined Size (Before Cleaning):** 1,063,106 rows

**Cleaning:** 91,300 rows containing NaN values (clean_title, 2_way_label) dropped from the combined post data.

**Final Cleaned Post Dataset Size:** 971,806 posts

### 3.2) Comment Processing

The bonus task involved enriching the feature space by incorporating comment threads, based on parent IDs (`t3_` for post, `t1_` for comment).

```
[+] Loading /kaggle/input/fakeddit-nlp-assignment/all_comments.tsv/all_comments.tsv

 [+] Required col found: id
 [+] Required col found: submission_id
 [+] Required col found: body
 [+] Required col found: parent_id
 [+] Loaded 10002177 comments.
 [+] Filtered 10000647 comments.
```

**Feature Engineering Logic:** A recursive function dynamically traverses the comment hierarchy for each post, creating a single, continuous string feature in the following format:

```
[COMMENT] This is the original comment.
[REPLY 1] Reply 1
[REPLY 1.1] Reply to Reply 1
[REPLY 2] Reply 2
```

### 3.3) Input Text Construction (Post-Comment-Reply Structure)

```
[+] Total rows in finalDF: 971806
[+] Rows with comments: 583299
[+] Rows without comments: 388507
```

The final dataset contains only three columns: **ID, text (post with comment threads) and label.**

**Dataset Link:** [Fakeddit Post-Comment Threads]

**Detailed Input Format Example:**

The core feature engineering step involved recursively traversing the comment hierarchy using **parent_id** indicators **(t3_ for top-level, t1_ for replies)** and concatenating all post titles, top-level comments, and replies into a single, comprehensive string. This structure allows the BERT model to treat the entire conversation as a continuous sequence, preserving contextual flow:

| Markers | Description | Example Value |
|---|---|---|
| [POST] | Marks the beginning of the original post title. | [POST] Why are fake posts more engaging? |
| [COMMENT] | Marks a top-level comment, nested immediately under the post. | [COMMENT] It's a controversy, always. |
| [REPLY 1] | Marks a nested reply, using decimal numbering for hierarchy. | [REPLY 1.1.1] Plus, anger drives shares. |

**[POST] Why are fake posts more engaging? (The post title)**

**[COMMENT] It's a controversy, always. (Top-Level Comment 1)**

**[REPLY 1] Exactly. Facts are boring. (Reply to Comment 1)**

**[REPLY 1.1] Plus, anger drives shares. (Reply to Reply 1.1)**

**[REPLY 2] I disagree, engagement is about novelty. (Second Reply to Comment 1)**

**[COMMENT] I think images matter more than text. (Top-Level Comment 2)**

**[REPLY 1] That's often true for viral content. (Reply to Comment 2)**

### 3.4) Distribution Statistics

**Note:** Due to the large size of the dataset, I couldn't train the model on the entire dataset. So I took 0.3 or 30% of the entire dataset to train and test the BERT Model which we call the balanced dataset.

| Statistic | Original Dataset (971,806 posts) | Balanced Dataset (291,541 posts) |
|---|---|---|
| **Fake Posts (1) Count** | 484,721 | 145,416 (30% sample) |
| **Real Posts (0) Count** | 487,085 | 146,125 (30% sample) |
| **Mean Comments (Fake)** | 5.50 | 5.56 |
| **Mean Comments (Real)** | 2.01 | 2.03 |
| **Std Dev Comments (Fake)** | 16.28 | 16.50 |
| **Std Dev Comments (Real)** | 6.74 | 6.82 |

A significant finding is the difference in comment activity between the classes: **fake posts generally attract more, or longer, conversational threads than real posts.** The balanced dataset was split using an 80 : 20 ratio, ensuring stratified sampling to maintain class balance in both partitions.

**Train Set Size: 233,232 posts**

**Test Set Size: 58,309 posts**

```
[>] Statistics Report

[+] Number of posts in original dataset: 971806
[+] Number of posts in balanced dataset (DF):
   [+] Real (0): 146125
   [-] Fake (1): 145416
   [=] Total :291541

[+] Distribution in train/test sets:
   [>] Train set (X): 233232 posts
   [>] Test set (y):  58309 posts

[+] Comment Statistics:
   [>] Total comments loaded (comments DF): 10000647
   [>] Total direct comments (comment_T1):  3866055
   [>] Total reply comments (comment_T2):   6134592

   [+] Posts with at least one comment (original): 583299
   [+] Mean + Standard Deviation of comments (Original Dataset):
       [>] Fake Posts: 5.50 mean / 16.28 std
       [>] Real Posts: 2.01 mean / 6.74 std

   [+] Mean + Standard Deviation of comments (Balanced Dataset):
       [>] Fake Posts: 5.56 mean / 16.50 std
       [>] Real Posts: 2.03 mean / 6.82 std
```

## 4) Task - 3: Fine-tuning pre-trained BERT Model

### 4.1) Training Hyperparameters

| Hyperparameter | Value | Description |
|---|---|---|
| **Model** | BertForSequenceClassification | Fine-tuned from bert-base-uncased. |
| **Optimizer** | AdamW | optimizer for training Transformer models |
| **Learning Rate** | 4e-5 | A typical value for Transformer fine-tuning. |
| **Epochs** | 3 | The no of complete passes over the training data. |
| **Batch Size** | 16 | The number of samples processed per update step. |
| **Max Token Length** | 512 | Input seq. padded or truncated to this length. |

### 4.2) Training Progress Summary

Training showed rapid convergence, achieving a high degree of classification accuracy within three epochs:

```
[+] Epoch 1 Summary:
  [>] Train Loss: 0.2269
  [>] Train Accuracy: 0.9136
  [>] Time: 12266.16s
```

```
[+] Epoch 2 Summary:
  [>] Train Loss: 0.1506
  [>] Train Accuracy: 0.9534
  [>] Time: 12275.74s
```

```
[+] Epoch 3 Summary:
  [>] Train Loss: 0.0904
  [>] Train Accuracy: 0.9768
  [>] Time: 12273.41s
```

## 4.3) Final Model Performance on Test Set (58,309 Samples)

The evaluation results on the test set demonstrate that the fine-tuned **BERT** model achieved a high and balanced classification performance.

**Test Accuracy:** 93.56%

This represents the overall percentage of correct predictions (both Real and Fake) out of all 58,309 test samples. The high value confirms the model's general effectiveness.

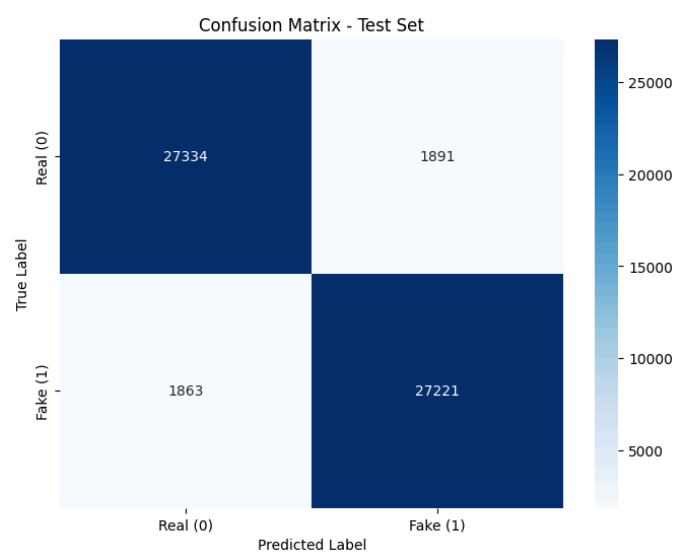
**Test Loss:** 0.2856

The final value of the cross-entropy loss on the test set. It indicates the residual error rate. This value is low and suggests the model has good generalization, though it is higher than the final training loss of 0.0904, which is expected due to regularization and preventing overfitting.

```
[!] --- Final Model Performance (Test Set) ---
[+]   Test Loss:     0.2856
[+]   Test Accuracy: 0.9356
[+]   Precision (for Fake):     0.9350
[+]   Recall (for Fake):        0.9359
[+]   F1-Score (for Fake):      0.9355
[!] ----------------------------------------

[+] Classification Report:
              precision    recall  f1-score   support

    Real (0)       0.94      0.94      0.94     29225
    Fake (1)       0.94      0.94      0.94     29084

    accuracy                           0.94     58309
   macro avg       0.94      0.94      0.94     58309
weighted avg       0.94      0.94      0.94     58309
```

**F1-Score of 0.94:** Since the F1-score is the harmonic mean of Precision and Recall, a high, consistent F1-score for both classes confirms the model performs well **without suffering from a class imbalance bias**. This validates the initial data engineering step of creating a balanced test set.

**Confusion Matrix:**




The nearly equal number of **FPs and FNs** further reinforces that the model is balanced and does not demonstrate a strong preference for classifying posts into one category over the other. The inclusion of the conversational context successfully trained the model to identify subtle linguistic patterns characteristic of fake news, resulting in powerful classification performance.